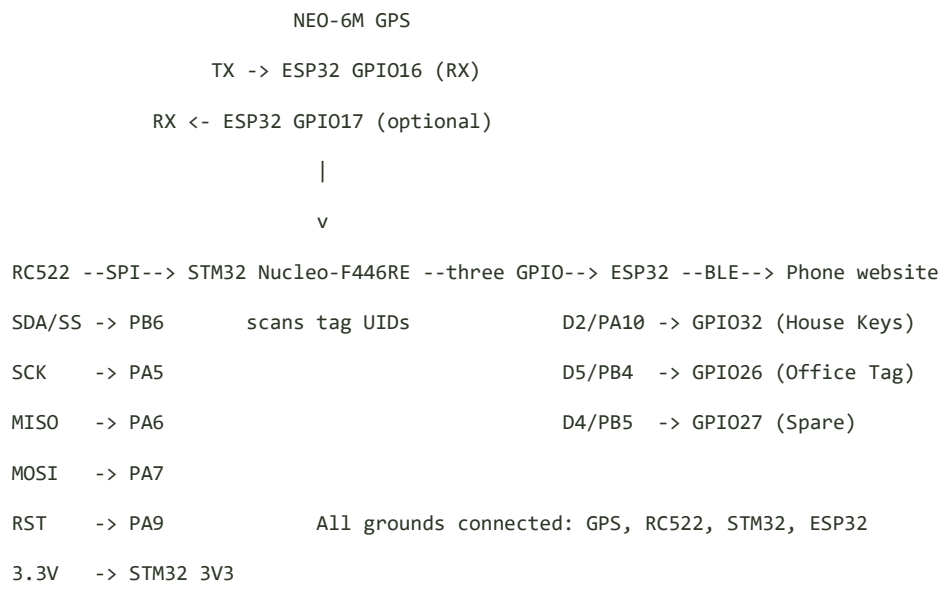


PurseGuard Hardware Wiring

STM32 Nucleo-F446RE + RC522 + ESP32 + NEO-6M GPS

Current design: the STM32 scans RFID tags over SPI and drives three active-high GPIO status signals to the ESP32. The ESP32 reads GPS over UART, samples the three tag signals, and publishes GPS and RFID over BLE.

System diagram



RC522 to STM32 SPI

RC522 pin	STM32 Nucleo-F446RE	Purpose
RC522 pin	STM32 Nucleo-F446RE	Purpose
SDA / SS	PB6 / Arduino D10	SPI chip select
SCK	PA5 / Arduino D13	SPI clock
MISO	PA6 / Arduino D12	Reader data to STM32
MOSI	PA7 / Arduino D11	STM32 data to reader
RST	PA9 / Arduino D8	Reader reset
3.3V	STM32 3V3	Power; RC522 is 3.3 V only
GND	STM32 GND	Common ground
IRQ	Not connected	Unused by this firmware

STM32 to ESP32 RFID status signals

Tag	STM32 output	ESP32 input
Tag	STM32 output	ESP32 input
House Keys	PA10 / Arduino D2	GPIO32

Office Tag	PB4 / Arduino D5	GPIO26
Spare	PB5 / Arduino D4	GPIO27
Ground reference	STM32 GND	ESP32 GND

NEO-6M GPS to ESP32

GPS module pin	ESP32 pin	Purpose
GPS module pin	ESP32 pin	Purpose
TX	GPIO16 (UART2 RX)	GPS data into ESP32; 9600 baud
RX	GPIO17 (UART2 TX)	Optional commands from ESP32
GND	GND	Join to the common ground
VCC	Supply supported by the GPS breakout	Check the breakout label; bare modules may not tolerate 5 V

Power, logic, and do-not-connect notes

- Join grounds across the GPS, RC522, STM32, and ESP32 so every signal has the same reference.
- STM32 status outputs are 3.3 V logic and connect directly to ESP32 inputs. Never feed 5 V into an ESP32 GPIO.
- Power the RC522 from 3.3 V only. GPS breakout supply tolerance varies by board; verify its markings.
- The old STM32 PA2 / Arduino D1 UART connection is not used in this GPIO design. Leave it disconnected from the ESP32.
- Leave STM32 PB3 / Arduino D3 unused; it may be used for SWO debug output.
- ESP32 GPIO27 is the Spare status input in the current firmware; it is not the old UART TX connection.

Data flow and firmware settings

RC522 reads tag UIDs -> STM32 compares the three configured UIDs -> STM32 D2/D5/D4 outputs presence levels -> ESP32 GPIO32/26/27 samples those levels -> BLE sends RFID,1,0,1 -> website updates the belongings list and alerts after 15 seconds missing.

Use `RFID_GPIO_TEST_MODE = 0U` for normal RC522 scanning. `RFID_UID_LEARN_MODE` defaults to `0U`; set it to `1U` temporarily to read a tag UID by itself in the STM32 SWV console. Update the UID table in `Src/main.c` if your physical tags differ.

The BLE RFID characteristic UUID is `cba1d466-344c-4be3-ab3f-189f80dd7518`. The ESP32 advertises as PurseGuard.